

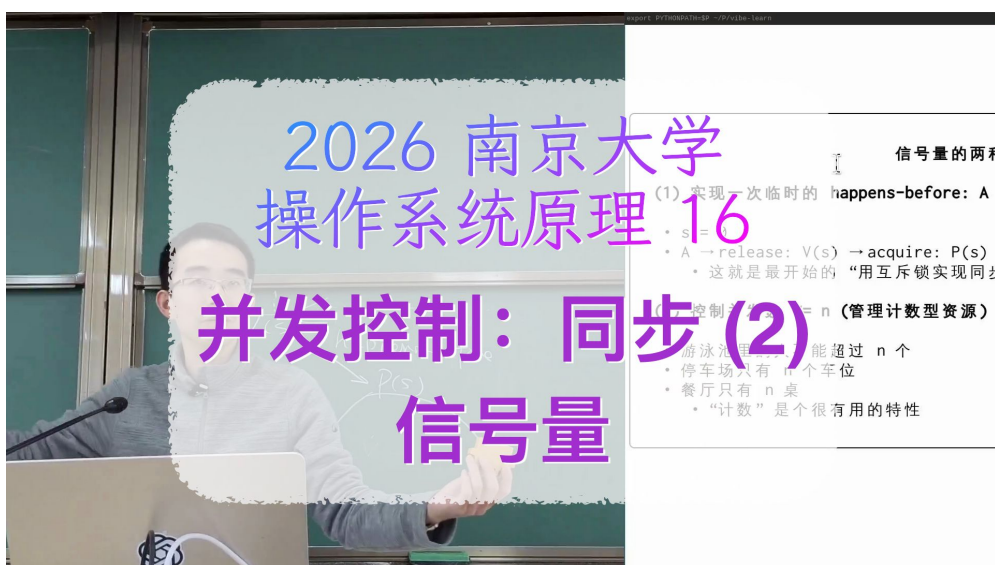
课程笔记

并发控制：信号量

南京大学 操作系统原理 2026 春季学期 第 16 讲

lecture-to-notes

2026 年 5 月 7 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 26 分钟

视频链接：<https://www.bilibili.com/video/BV1yQogB2Esf>

目录

1 回顾与动机	2
1.1 从 DAG 调度看条件变量的“☹️”	2
1.2 大胆设想：用互斥锁直接同步？	3
1.3 本章小结	4
2 发明信号量：P/V 操作模型	4
2.1 Dijkstra 的 P/V 操作	4
2.2 信号量是互斥锁的扩展	5
2.3 信号量 killer app：优雅的生产者-消费者	6
2.4 用信号量实现 happens-before	7
2.5 本章小结	7
3 信号量 vs 条件变量：哲学家就餐问题	8
3.1 问题描述	8
3.2 解法 1：条件变量（万能同步方法）	8
3.3 解法 2 朴素版：每把叉子一个信号量（错误!）	9
3.4 解法 2 修补版：固定上锁顺序（self-organize）	10
3.5 解法 3：服务员/消息驱动（waiter pattern）	11
3.6 三种解法对比	11
3.7 本章小结	11
4 总结与延伸	12
4.1 讲师的核心论断	12
4.2 我的结构化提炼	13
4.3 开放问题与下一讲铺垫	13
4.4 拓展阅读	13

1 回顾与动机

上一讲发明了条件变量，并提出了”万能同步方法”：把任意同步问题翻译成生产者-消费者，用 `mutex + while + cond_wait + broadcast` 几行模板就能写正确。本讲要回答两个新问题：

- 条件变量真的”万能”吗？它当然能写对，但有些场景写起来很🐱🐼
- 有没有一种更简洁的同步原语？——信号量（semaphore）

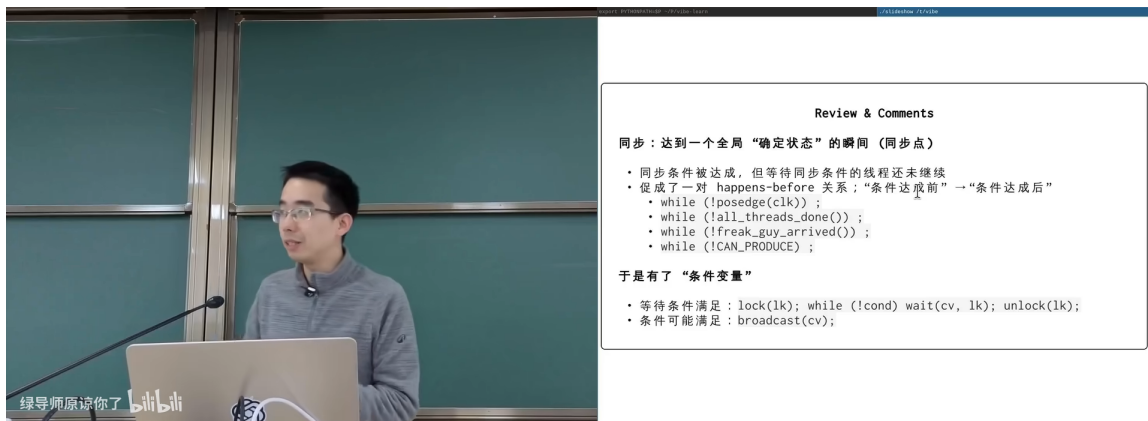


图 1: 开场回顾：上一讲的同步条件、生产者-消费者、while 循环 + cond_wait + broadcast 模板¹

1.1 从 DAG 调度看条件变量的”🐱🐼”

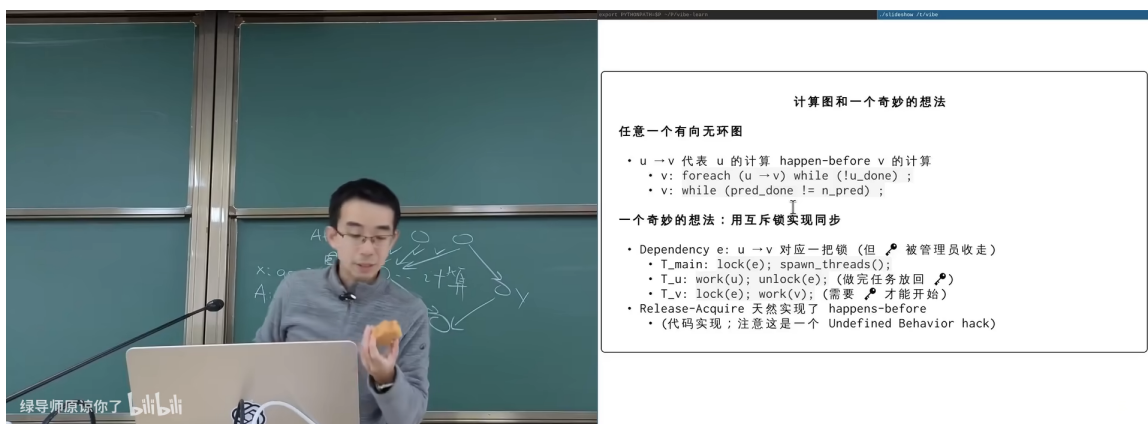


图 2: 回顾：用条件变量实现 DAG 计算图调度——每个节点都要写 `while !ready: cond_wait`²

回顾上节的 DAG 调度，每个节点的代码大致是：

```
1 mutex_lock(&v->m);
2 while (v->n_pending_deps > 0)
3     cond_wait(&v->ready_cv, &v->m);
4 mutex_unlock(&v->m);
5 compute(v);
```

¹ 视频画面时间区间：00:01:00-00:01:30。讲师强调：“在你任何时候同步条件可能发生改变的时候，都需要执行一个 `broadcast`。”

² 视频画面时间区间：00:08:00-00:08:30。

```

6 for (Node *succ : v->successors) {
7     mutex_lock(&succ->m);
8     succ->n_pending_deps--;
9     if (succ->n_pending_deps == 0)
10         cond_broadcast(&succ->ready_cv);
11     mutex_unlock(&succ->m);
12 }

```

Listing 1: 用条件变量实现的 DAG 节点（上一讲的方案）

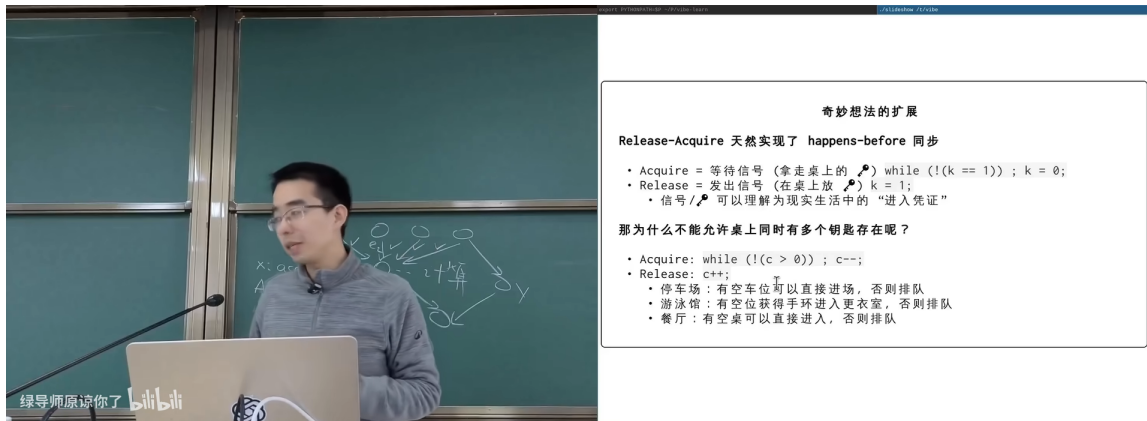
有没有更优雅的写法？

注意每个节点都需要：

- 一把 mutex
- 一个 cond
- 一个 n_pending_deps 计数器
- 通知后继时显式 lock → counter-- → broadcast → unlock

这套机制能用更少的概念表达吗？讲师由此引出今天的核心想法。

1.2 大胆设想：用互斥锁直接同步？



奇妙想法的扩展

Release-Acquire 天然实现了 happens-before 同步

- Acquire = 等待信号 (拿走桌上的 ) while (!(k == 1)); k = 0;
- Release = 发出信号 (在桌上放 ) k = 1;
- 信号 /  可以理解为实现生活中的“进入凭证”

那为什么不能允许桌上同时有多个钥匙存在呢？

- Acquire: while (!(c > 0)); c--;
- Release: c++;
- 停车场：有空车位可以直接进场，否则排队
- 游泳馆：有空位获得手环进入更衣室，否则排队
- 餐厅：有空桌可以直接进入，否则排队

图 3: 聪明的想法：每条依赖边放一把锁，main 先 lock 全部，前驱算完后 unlock，后继再 acquire ——把同步压缩成”传递钥匙”⁴

⁴视频画面时间区间：00:09:00–00:11:00。

跨线程的”锁交接”也能实现同步

对每条依赖边 $a \rightarrow x$:

1. **main** 主线程在所有 **worker** 启动前，先把这把锁 lock 住（”把玩具没收到桌子下”）
2. 节点 **a** 算完后 unlock 这把锁（”把玩具放回桌上”）
3. 节点 **x** 启动时试图 lock 这把锁——拿不到就阻塞，拿到说明 **a** 已完成

讲师的金句：”上节课说锁不能用来同步，是有前提的——前提是 *lock/unlock* 在同一个线程上配对。如果允许跨线程交接，*token* 的传递本身就可以实现 *happens-before*。”

但这个用法很别扭——**我们是在滥用 mutex 的名字**。能不能给”跨线程传递的钥匙”起一个干净的名字？答案就是**信号量**。

1.3 本章小结

- 条件变量万能但罔噤：每个同步点都要 `mutex + cv + counter`
- 跨线程”传递锁”也能实现 *happens-before*
- 把”传递钥匙”提炼为新原语 → 信号量

2 发明信号量：P/V 操作模型

2.1 Dijkstra 的 P/V 操作

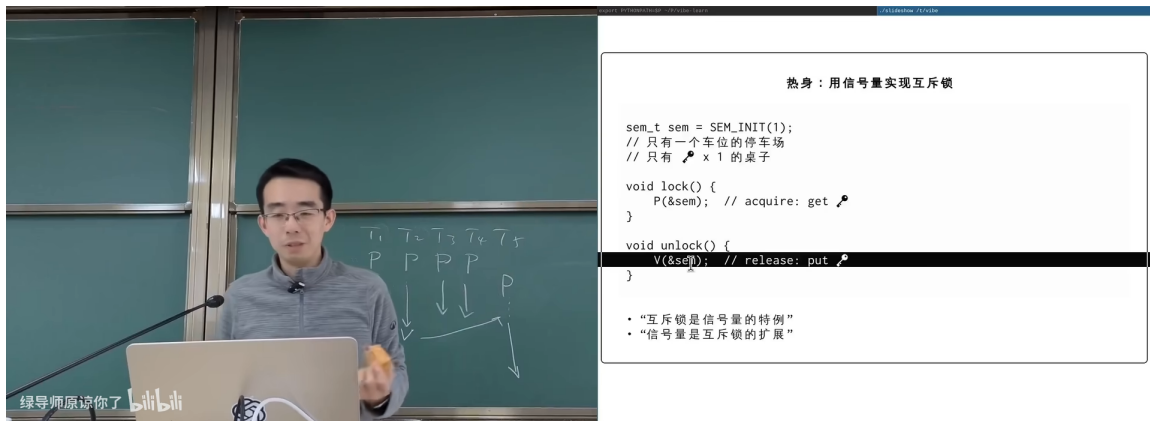


图 4: 信号量物理模型：桌上一个袋子里有若干个玩具/钥匙；P 操作拿走一个、V 操作放回一个⁵

⁵视频画面时间区间：00:23:00–00:23:30。

信号量

信号量 (semaphore) 是一个非负整数计数器 `count`，配两个原子操作（来自荷兰语，Dijkstra 在 1965 年提出）：

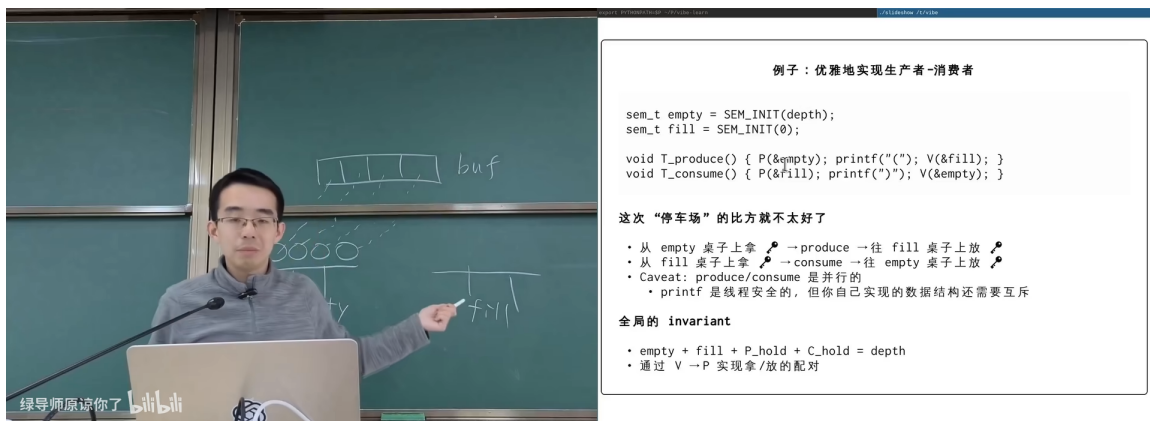
- **P(s)** (*passeren*, 过/拿)：等到 `count > 0`，然后 `count--`（拿走一个玩具）。也叫 `wait` / `acquire` / `down`
- **V(s)** (*vrijgeven*, 释放)：`count++`（放回一个玩具）。也叫 `post` / `signal` / `release` / `up`

POSIX API: `sem_init` / `sem_wait` / `sem_post` / `sem_destroy`。`sem_init(&s, 0, n)` 把袋子里初始放 `n` 个玩具。

物理直觉：

- 把信号量想象成一个袋子（或一张桌子，或一个停车场）
- 袋子里有若干个相同的小玩具（钥匙、车位、苹果...）
- P 操作 = 从袋子里取走一个；袋子空了就阻塞
- V 操作 = 往袋子里放入一个；任何线程都能放

2.2 信号量是互斥锁的扩展



例子：优雅地实现生产者-消费者

```
sem_t empty = SEM_INIT(depth);
sem_t fill = SEM_INIT(0);

void T_produce() { P(&empty); printf(""); V(&fill); }
void T_consume() { P(&fill); printf(""); V(&empty); }
```

这次“停车场”的比方就不太好了

- 从 empty 桌子上拿 → produce → 往 fill 桌子上放
- 从 fill 桌子上拿 → consume → 往 empty 桌子上放
- Caveat: produce/consume 是并行的
 - printf 是线程安全的，但你自己实现的数据结构还需要互斥

全局的 invariant

- `empty + fill + P_hold + C_hold = depth`
- 通过 `V → P` 实现拿/放的配对

图 5: 互斥锁 = 计数为 1 的信号量（“只有一个车位的停车场”）⁶

⁶视频画面时间区间：00:24:00-00:24:30。

用信号量实现互斥锁

初始化 `count == 1`: 袋子里只有一把钥匙——任何时刻最多一个线程能进入临界区。

```
1 sem_t lock;
2 sem_init(&lock, 0, 1); /* 一个车位的停车场 */
3
4 void critical() {
5     sem_wait(&lock);    /* 取钥匙; 取不到就阻塞 */
6     /* critical section */
7     sem_post(&lock);    /* 还钥匙 */
8 }
```

Listing 2: 用信号量做互斥锁

但是要注意一个微妙的差异: 互斥锁严格规定 lock/unlock 必须配对 (同一线程上锁、同一线程解锁); 而信号量没这个限制——任何线程都可以 V 操作”凭空变出一个车位”。如果你写代码时想当然地连续 `sem_post` 两次, 停车场就从 1 个车位变成 2 个, 互斥性被破坏。信号量给了你更大的自由, 也给了你更大的犯错空间。

2.3 信号量 killer app: 优雅的生产者-消费者

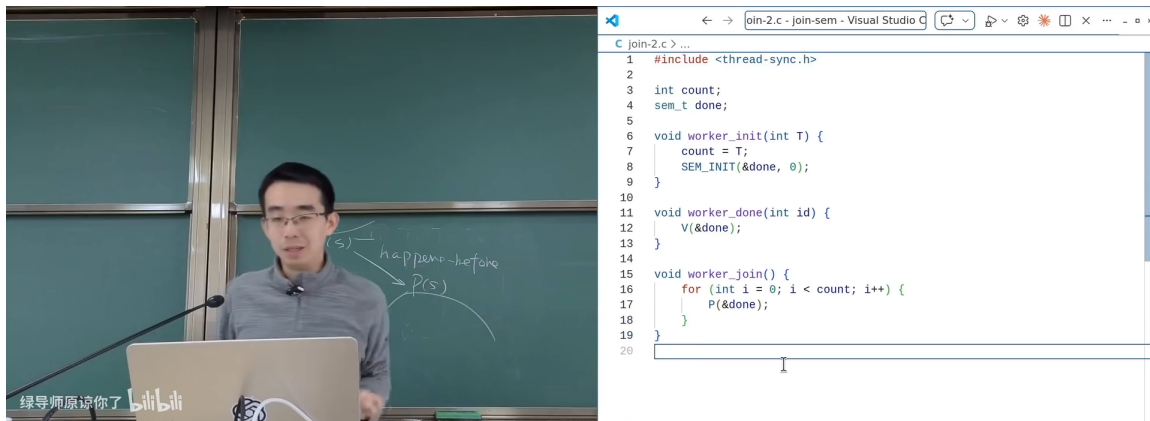


图 6: 用 `empty` 和 `full` 两个信号量实现 P/C, 对称如诗⁷

容量 `N` 的有界缓冲区, 两个信号量分别表示”还能放进去多少个”和”已经有多少可以取”:

```
1 sem_t empty; /* 还能放进多少 */
2 sem_t full; /* 已经有多少可以取 */
3 sem_init(&empty, 0, N); /* 一开始 N 个空位 */
4 sem_init(&full, 0, 0); /* 一开始没有可取的 */
5
6 void producer() {
7     sem_wait(&empty); /* 占一个空位 */
8     put_into_buffer();
9     sem_post(&full); /* 多了一个可取 */
10 }
```

⁷ 视频画面时间区间: 00:35:00–00:35:30。讲师评价: ”极为对称、特别优雅。”

```

11 void consumer() {
12     sem_wait(&full);    /* 占一个 ready 项 */
13     take_from_buffer();
14     sem_post(&empty);   /* 多了一个空位 */
15 }

```

Listing 3: 信号量版生产者-消费者（极简对称）

为什么这段代码”对称如诗”？

- Producer 和 Consumer 是镜像：P(empty) ... V(full) 对应 P(full) ... V(empty)
- 没有显式 mutex（多 producer/consumer 时仍然要 mutex 保护 buffer 数据结构本身，但容量计数已经被信号量包办）
- 没有 while 循环：sem_wait 内部会自旋/睡眠到 count > 0
- 没有 condition variable：empty/full 直接编码同步语义

思维方式的根本差异：cv 关心的是”条件何时为真”，sem 关心的是”资源还剩多少”。当问题本身是资源计数（车位、缓冲槽、令牌）时，sem 更直接。

2.4 用信号量实现 happens-before

更深一层地看，信号量是表达 happens-before 关系最简洁的工具：

```

1 sem_t s;
2 sem_init(&s, 0, 0);    /* 初始为 0 */
3
4 /* 线程 A */           /* 线程 B */
5 do_X();                sem_wait(&s);
6 sem_post(&s);          do_Y();

```

Listing 4: 信号量编码 happens-before

- B 的 sem_wait 必须等到 A 的 sem_post
- 所以 A 中 do_X() happens-before B 中 do_Y()
- 这就是为什么用信号量做 DAG 调度只需”每条边一个信号量”，比 cv 版精简一半

2.5 本章小结

- 信号量 = 计数器 + P/V 原子操作
- 互斥锁 = 计数为 1 的信号量
- 任何 happens-before 都可以用 P + V 编码
- 生产者-消费者用 empty + full 两个信号量优雅实现

3 信号量 vs 条件变量：哲学家就餐问题

3.1 问题描述

Dijkstra 的哲学家就餐问题

五位哲学家围圆桌，每人面前一盘意面，相邻两人之间有一把叉子。哲学家轮流思考与吃饭。
吃饭必须同时拿起左右两把叉子。

要求：写一个并发程序，让哲学家们都能吃上饭、不会饿死。

这个问题的妙处：每个并发原语在它身上的”优雅度 / 正确性 / 性能” 权衡都暴露得很清楚。

3.2 解法 1：条件变量（万能同步方法）

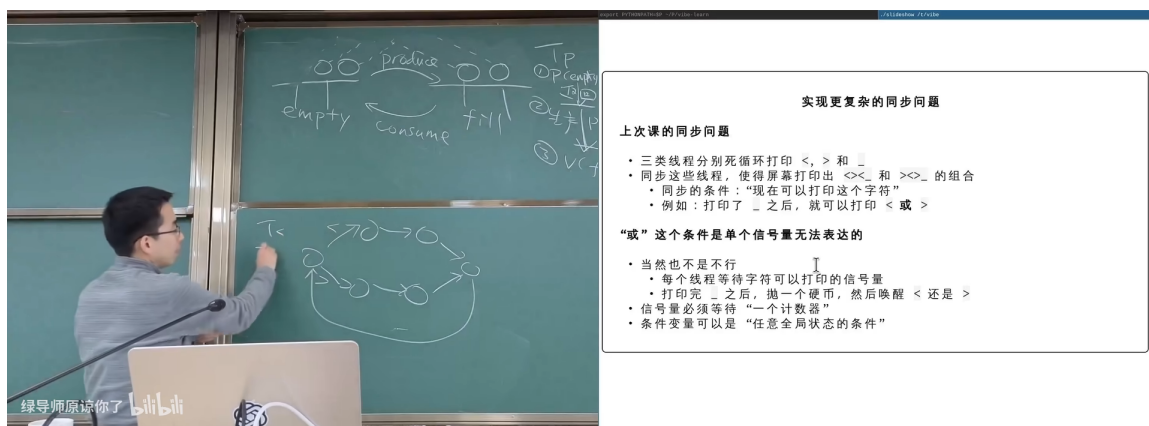


图 7：条件变量解：每位哲学家等”左右两把叉子都空”再拿起——直接表达同步条件⁸

```
1 mutex_t m;  
2 cond_t cv;  
3 int avail[N] = {1,1,1,1,1}; /* 每把叉子是否可用 */  
4  
5 void philosopher(int i) {  
6     int L = i, R = (i+1) % N;  
7     while (1) {  
8         mutex_lock(&m);  
9         while (!(avail[L] && avail[R]))  
10             cond_wait(&cv, &m);  
11         avail[L] = avail[R] = 0;  
12         mutex_unlock(&m);  
13  
14         eat();  
15  
16         mutex_lock(&m);  
17         avail[L] = avail[R] = 1;  
18         cond_broadcast(&cv);  
19         mutex_unlock(&m);  
20     }
```

⁸视频画面时间区间：00:53:00-00:54:30。讲师称这是 ”两分钟就能写、期末考试值 20 分” 的代码。

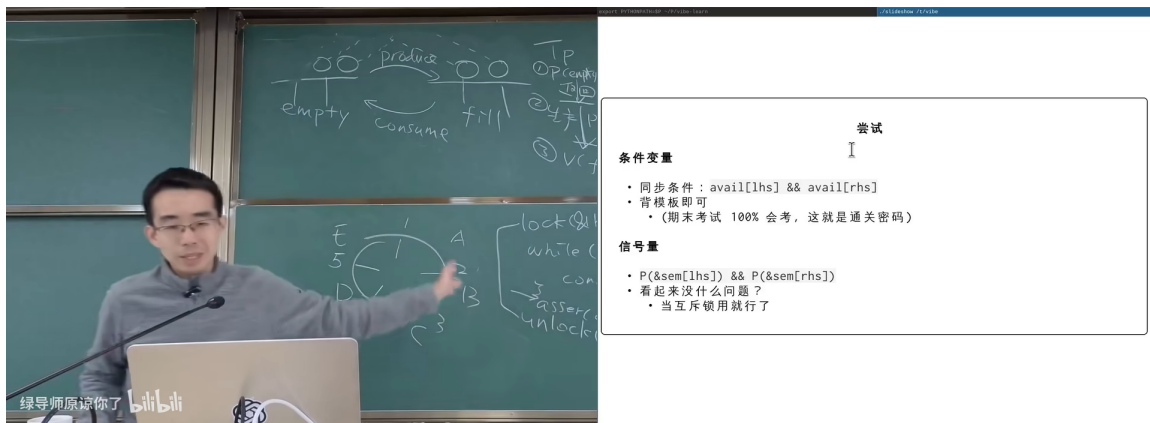
Listing 5: 条件变量版的哲学家就餐

为什么这版 一定对？

- 临界区有锁保护，`avail` 数组的检查与修改是原子的
- `while + cond_wait` 防假唤醒
- `broadcast` 在状态改变时叫醒所有可能受益者
- 无死锁：临界区内一次性看到全局，要么两叉子都拿到要么继续等

性能不是最快，但 *simple, clean, correct* 三件齐全。讲师反复强调：“你们任何时候设计系统，首先要遵循的是简单、干净、正确的方案。性能优化是另一件事。”

3.3 解法 2 朴素版：每把叉子一个信号量（错误!）

图 8: 朴素信号量解：每把叉子一个信号量（计数 1），先 P 左、再 P 右——会死锁⁹

```

1 sem_t fork_sem[N];
2 for (int i=0;i<N;i++) sem_init(&fork_sem[i], 0, 1);
3
4 void philosopher(int i) {
5     int L = i, R = (i+1) % N;
6     while (1) {
7         sem_wait(&fork_sem[L]); /* 拿左 */
8         sem_wait(&fork_sem[R]); /* 拿右 */
9         eat();
10        sem_post(&fork_sem[L]);
11        sem_post(&fork_sem[R]);
12    }
13 }

```

Listing 6: 朴素信号量版（错!）

⁹视频画面时间区间：00:57:00-00:58:00。讲师演示：所有人同时拿左手时全部卡死。

为什么死锁？

若所有 5 位哲学家同时各自拿到左手叉子，那么每个人手上都拿着一把、又都在等右手——形成环形等待（成环依赖）。所有线程都阻塞在 `sem_wait(&fork_sem[R])`，永远拿不到，谁也吃不上。这就是经典死锁。

死锁的四个必要条件（下一讲细讲）：

1. 互斥占用——叉子是排他的
2. 持有并等待——拿着左手等右手
3. 不可抢占——拿到了就不会被别人抢走
4. 环路等待——哲学家之间形成圆环

任意打破一个就能避免死锁。

3.4 解法 2 修补版：固定上锁顺序 (self-organize)

最简单的修补：打破环路等待——所有线程按相同的全局顺序申请资源。

```
1 void philosopher(int i) {
2     int L = i, R = (i+1) % N;
3     int low = (L < R) ? L : R;
4     int high = (L < R) ? R : L;
5     sem_wait(&fork_sem[low]);    /* 永远先拿编号小的 */
6     sem_wait(&fork_sem[high]);
7     eat();
8     sem_post(&fork_sem[high]);
9     sem_post(&fork_sem[low]);
10 }
```

Listing 7: 按编号顺序拿叉子（破坏）

- 优点：完全去中心化，每个线程自治，性能高（高并发下可同时多人吃饭）
- 缺点：必须所有用户线程严格遵守上锁顺序，是个分布式约定，规模大时容易出错

3.5 解法 3: 服务员/消息驱动 (waiter pattern)

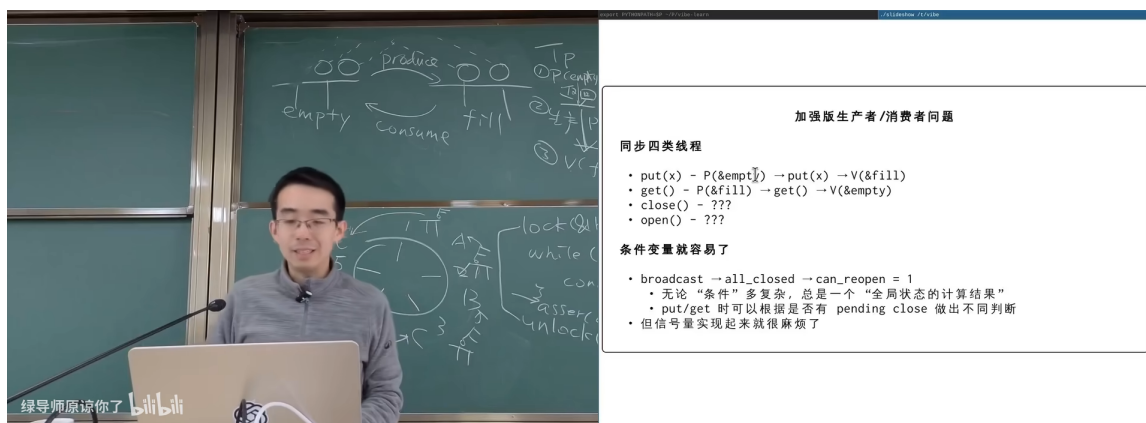


图 9: Waiter 模式：哲学家把”我要吃饭”作为消息发给一个集中调度者；服务员串行处理请求¹⁰

消息驱动

引入一个**服务员 (waiter)** 线程，独占叉子的全局状态。哲学家通过**消息队列**与服务员通信：

- Philosopher: `send(want_eat); recv(grant);`
- Waiter: `while(1) { msg = recv(); if (forks_available(msg)) grant(msg); }`

消息队列本身就是一个生产者-消费者，用两个信号量实现。这样：

1. **服务员的逻辑是串行的**：他在自己的本地状态里检查叉子，没有并发竞争 → 逻辑超级简单
2. **中心化的调度可以做策略**：公平性（先到先得）、负载均衡、限流
3. **所有同步问题都成了 producer-consumer**（再次回到上节的”万能同步方法”）

讲师的总结：“自组织协议很 cool，但消息驱动更通用、更可维护，几乎所有的并发框架 (Erlang/Go channel/actor model) 都是这个思路。”

3.6 三种解法对比

解法	思路	死锁风险	性能	可维护性
条件变量	全局 mutex + 检查同步条件	无	中 (全局锁)	高
朴素 sem	每叉子一信号量，按手序	有	高	低
sem + 序	按编号统一上锁顺序	无	高	中 (约定)
Waiter	集中调度 + 消息队列	无	中	高

3.7 本章小结

- 同一问题，cv 直接但啰嗦、sem 简洁但易死锁

¹⁰ 视频画面时间区间：01:19:00–01:21:00。

- 朴素 sem 用法常常有环路等待，要靠”协议”破坏
- Waiter 模式把并发压成串行调度——是工业界框架的主流路径
- 对错优先于快：先写正确简单的版本，再针对瓶颈优化

4 总结与延伸

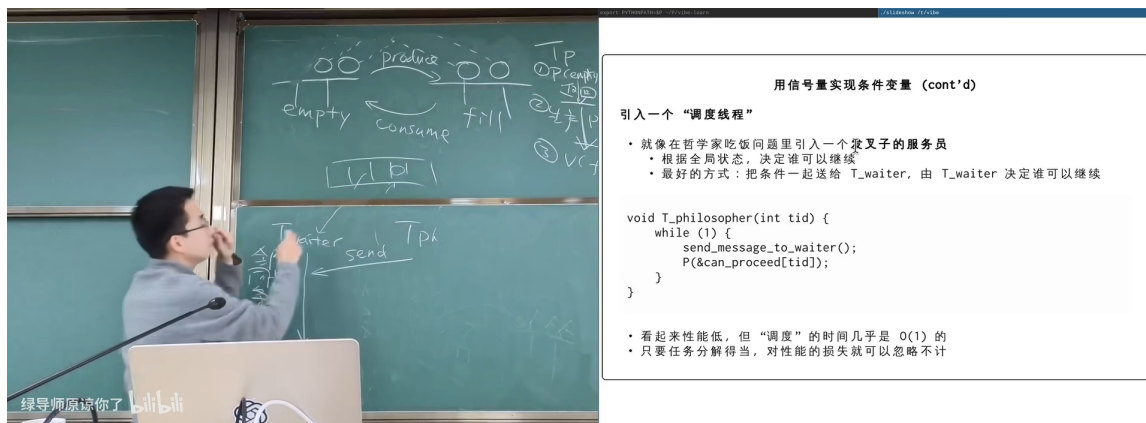


图 10: 本节课的并发原语全景图：从 mutex / cv / sem 到 message queue / actor¹¹

4.1 讲师的核心论断

两条主线

主线 1：信号量 = mutex 的扩展。

- count == 1 的信号量退化为 mutex
- 跨线程的 P/V 直接表达 happens-before
- 生产者-消费者用 empty + full 两个信号量极简对称

主线 2：cv 与 sem 的取舍是抽象层次的取舍。

- cv 直接对应”等条件成立” → 思考清晰、无死锁风险，但代码量大
- sem 直接对应”资源计数” → 代码简洁、但容易踩死锁
- 把同步问题翻成 producer-consumer + waiter，能消除大多数环形依赖

¹¹ 视频画面时间区间：01:23:00–01:24:00。

4.2 我的结构化提炼

	场景	推荐原语
何时用什么	保护数据结构（互斥）	<code>mutex</code> （或 <code>count=1</code> 的 <code>sem</code> ）
	等任意条件成立	<code>condition variable</code> + <code>while loop</code>
	有界缓冲区 / 资源池	信号量（ <code>empty</code> + <code>full</code> ）
	跨线程 <code>happens-before</code>	单计数信号量
	复杂同步 / 公平性 / 限流	<code>waiter</code> 模式 + 消息队列

学完信号量后要相信的事

1. 任何同步问题都至少有 **cv** 和 **sem** 两条独立解法，可以互相验证
2. 朴素 `sem` 写并发常常死锁，**破坏**比避坑重要
3. **消息驱动 + 串行调度器**是大型并发系统的”政治正确”答案
4. **先写对再写快**（讲师反复强调）

4.3 开放问题与下一讲铺垫

- 哲学家就餐展示了**死锁**，下一讲（第 17 讲）会专门讲死锁的四个必要条件、检测、避免、恢复
- 数据竞争（`race condition`）、原子性违反（`atomicity violation`）、顺序违反（`order violation`）也是下一讲的重点
- 工程上常用的并发 bug 检测工具：`ThreadSanitizer`、`lockdep`、`Helgrind`、`Eraser`

4.4 拓展阅读

- Dijkstra, "Cooperating Sequential Processes," 1965（信号量原文）
- OSTEP 第 31 章 Semaphores
- Hoare, "Communicating Sequential Processes," 1978（CSP / Go channel 的源头）
- Lamport, "Time, Clocks, and the Ordering of Events," 1978（`happens-before` 的形式化）